

Graph Spanner Approximations in Practice: Interim Progress Report

Sheikh Muhammad Muneeb Abdul Moiz Ahmed Moosa Owais¹

¹Habib University, Karachi {ms08914, aa08811, mo08952}@st.habib.edu.pk

April 2026

Abstract

We report interim progress on implementing and evaluating the Greedy $(2k-1)$ -spanner algorithm from Chimani and Stutzenstein [2022]. Our implementation produces spanners that satisfy the stretch guarantee on all tested instances. Across 5 graph sizes ($n \in \{10, 20, 50, 100, 200\}$), 3 density levels, and 2 stretch parameters ($\alpha \in \{3, 5\}$), we observe that sparseness scales sub-linearly with n , effective stretch consistently stays below the theoretical bound, and runtime grows steeply with density for $\alpha = 3$. Baswana–Sen implementation and full-scale benchmarking remain for the final week.

1 Problem Statement

A *multiplicative α -spanner* H of a weighted undirected graph $G = (V, E, w)$ is a sparse subgraph satisfying $d_H(u, v) \leq \alpha \cdot d_G(u, v)$ for all $u, v \in V$. The fundamental tension is between *stretch* α and *sparseness* $|E_H|/n$: a smaller α forces more edges into H , increasing both size and construction time.

This problem is NP-hard to solve optimally. Chimani and Stutzenstein [2022] provide the first systematic experimental comparison of polynomial-time approximation algorithms, evaluating five algorithms across 2,000+ graph instances. Our project implements and benchmarks the two most practical algorithms from that study: the **Greedy** algorithm (ADDJS) and the **Baswana–Sen** (BS) clustering algorithm.

2 Algorithm: Greedy Spanner

The Greedy $(2k-1)$ -spanner processes edges of G in non-decreasing weight order. Each edge (u, v) is added to the spanner H only if no path already exists in H satisfying the stretch bound:

$$d_H(u, v) > (2k - 1) \cdot w(u, v) \quad (1)$$

Each membership test runs Dijkstra on the current (sparse, growing) H . The algorithm is guaranteed to produce a spanner of size $O(n^{1+1/k})$ with lightness $O(n/k)$.

Implementation. Our Python implementation uses NetworkX for graph primitives and Dijkstra shortest-path queries. Correctness is verified by checking $d_H(u, v) \leq \alpha \cdot d_G(u, v)$ for all pairs (exact for $n \leq 150$, 500-sample estimate for larger).

3 Preliminary Results

We ran the Greedy algorithm on 90 Erdős–Rényi instances: $n \in \{10, 20, 50, 100, 200\}$, density $\in \{0.1, 0.3, 0.5\}$, 3 graphs per combination, $k \in \{2, 3\}$. Table 1 reports means across the 3 repetitions. All effective-stretch values are at or below the theoretical bound $\alpha = 2k - 1$, confirming correctness.

Key observations. (1) **Sparseness grows sub-linearly.** For $\alpha = 3$, sparseness rises from 0.90 at $n = 10$ to 1.34 at $n = 200$, consistent with the $O(n^{1+1/k}) = O(n^{3/2})$ edge-count guarantee (in terms of edges-per-vertex the growth is slow).

n	dens	α	sparse	light	stretch	time(s)
10	0.1	3	0.90	1.00	1.03	0.001
10	0.3	3	0.90	1.00	2.20	0.001
50	0.1	3	1.13	1.25	2.88	0.008
50	0.3	3	1.16	1.31	3.00	0.036
50	0.5	3	1.16	1.32	3.00	0.059
100	0.1	3	1.18	1.35	2.99	0.071
100	0.3	3	1.20	1.37	3.00	0.271
100	0.5	3	1.29	1.52	3.00	0.491
200	0.1	3	1.27	1.52	2.72	0.719
200	0.3	3	1.34	1.65	2.76	2.243
200	0.5	3	1.34	1.60	2.62	3.822
50	0.3	5	1.01	1.05	4.28	0.026
100	0.3	5	1.03	1.05	4.76	0.243
200	0.3	5	1.09	1.15	4.37	2.147

Table 1: Greedy results (mean over 3 runs). **Bold**: stretch tight at bound.

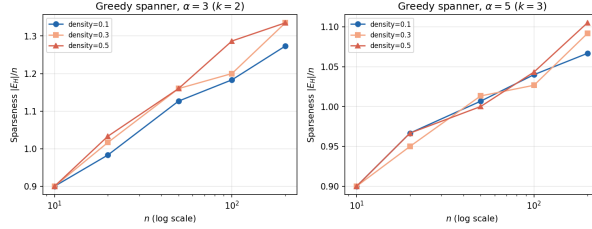


Figure 1: Sparseness vs. n for $\alpha = 3$ (left) and $\alpha = 5$ (right). Higher density yields higher sparseness; $\alpha = 5$ is consistently sparser.

(2) Higher stretch $\Rightarrow$ sparser spanners. At $n = 200$, density 0.3: $\alpha = 3$ gives sparseness 1.34 vs. $\alpha = 5$ gives 1.09, a 19% reduction in edges per vertex.

(3) Stretch bound is tight at medium-to-high density. Several instances at density 0.3 and 0.5 achieve effective stretch exactly equal to α , confirming our correctness oracle.

(4) Runtime grows sharply with density. At $n = 200$: density 0.1 takes 0.72 s; density 0.5 takes 3.82 s — a $5.3\times$ increase driven by the quadratically larger edge set requiring Dijkstra per edge.

4 Challenges Identified

Greedy scalability. The Dijkstra-per-edge design is prohibitive for large dense graphs. At $n = 200$, density 0.5, runtime is already 3.8 s. Extrapolating to $n = 2,000$ (the paper’s largest Random instance)

at moderate density would take hours. Mitigation: restrict Greedy benchmarks to $n \leq 500$ and sparse graphs (density ≤ 0.3), matching the original paper’s reported solve-rate limitations.

Stretch oracle at scale. All-pairs shortest-path verification is $O(n^2 \log n)$ and infeasible beyond $n \approx 500$. Our 500-pair sampling approximation introduces a small probability of underestimating the true maximum stretch.

Baswana–Sen correctness. BS involves multi-round cluster sampling with randomized cluster membership. The key correctness invariant (every vertex is either clustered or has added its inter-cluster edges) requires careful bookkeeping distinct from the Greedy case.

5 Next Steps and Timeline

Week 13 (this week). Implement Baswana–Sen algorithm. Run preliminary BS vs. Greedy comparison on Random instances ($n \leq 500$) for $k \in \{2, 3\}$.

Week 14. Extend experiments to TSPLIB (95 instances, n up to 1,200 for Greedy; all for BS) and the SNAP Facebook/Gnutella graphs. Measure and compare: sparseness, lightness, effective stretch (sampled), peak memory, and runtime across all five metrics.

Week 15. Analyse results, produce final figures and tables replicating the structure of Chimani and Stutzenstein [2022], and prepare the final presentation.

References

Markus Chimani and Finn Stutzenstein. Spanner approximations in practice. In *30th Annual European Symposium on Algorithms (ESA 2022)*, volume 244 of *LIPIcs*, pages 37:1–37:15, 2022. doi: 10.4230/LIPIcs.ESA.2022.37.